



United International University
QUEST FOR EXCELLENCE



**CSI 121: Structured Programming
Language
Simple Functions
Salekul Islam
United International University**

Outline

- **Functions**

- Defining a Function
- Arguments and Local Variables
- Returning Function Results
- Declaring a Function Prototype
- Global Variables

What is a function

- A function in C: is a self-contained unit of program code designed to accomplish a particular task.
- The concept has some equivalent in all high-level programming languages: *functions*, *subroutines*, and *procedures*
- The use of a function: a "black box"
 - defined in terms of the information that goes in (its input) and the value or action it produces (its output).
 - what goes on inside the black box is not your concern, unless you are the one who has to write the function.
 - Think on how you used functions printf, scanf, getchar !
- What kind of "output" comes out from a function black box ?
 - Some functions find a **value** for a program to use. Example: getchar() returns to the program the next character from the standard input buffer.
 - Some functions cause an **action** to take place. Example: printf() causes data to be printed on the screen
 - In general, a function can both produce actions and provide values.

Defining a function

```
#include <stdio.h>
```

```
void printMessage (void)
{
    printf ("Programming is fun.\n");
}
```

*Function
Definition*

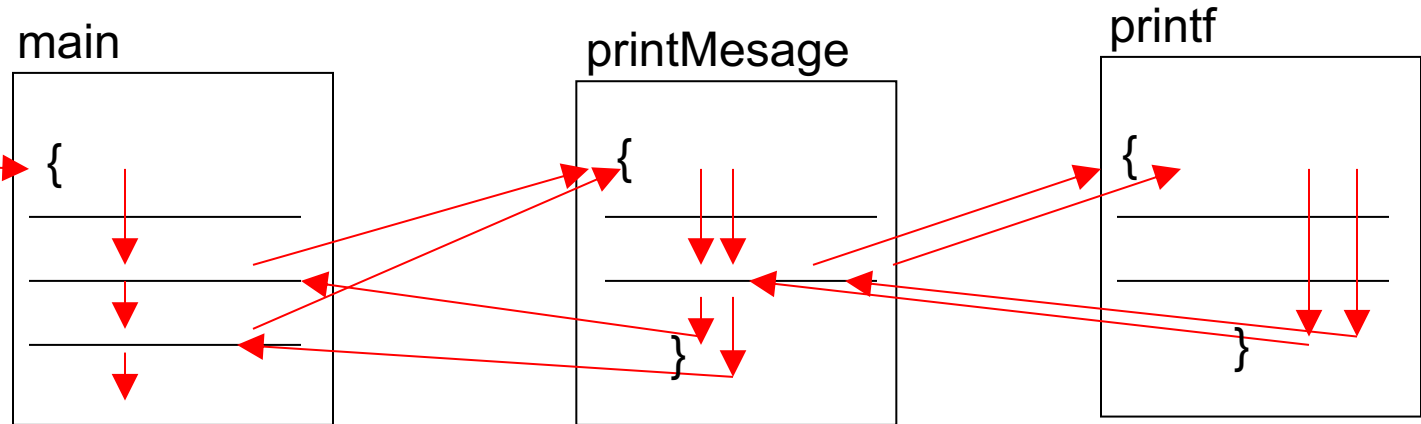
*-occurs ONE time for all
-outside other functions*

```
int main (void)
{
    printMessage ();
    printMessage ();
    return 0;
}
```

*Function
calls (invocations)*

*-occurs ANY (0-N) times
-statement inside (other) functions body*

Transfer of control flow



When a function call is executed, program execution is transferred directly to the indicated function. After the called routine is finished (as signaled by the closing brace) the program *returns* to the calling routine, where program execution continues at the point where the function call was executed.

Find the output:

```
#include<stdio.h>
void func(int x);
int main()
{
    func(3);
    return 0;
}
```

```
void func(int x)
{
    if (x==0)
        return;
    printf("%d\n", x);
    func(x-1);
    printf("%d\n", x);
}
```

Function definitions

General form of function definition:

```
return-type function-name(argument declarations)  
{  
    declarations and statements  
}
```

return-type

arguments

```
void printMessage ( void )
```

```
{
```

```
    printf ("Programming is fun.\n");
```

```
}
```

Function prototype

- The first line of the function definition
- Contains everything that others (other functions) need to know about the function in order to use it (call it)
- `void printMessage (void)`
- `void calculateTriangularNumber (int n)`

Function prototype

```
return-type function-name(argument declarations)
```

```
{
```

```
declarations and statements
```

```
}
```

Function arguments

- arguments (parameters): a kind of input for the function blackbox
- **In the function definition:** *formal arguments* (formal parameters)
 - Formal parameter: a name that is used inside the function body to refer to its argument
- **In the function call:** *actual arguments* (actual parameters)
 - The actual arguments are values are assigned to the corresponding formal parameters.
 - The actual argument can be a constant, a variable, or an even more elaborate expression.
 - The actual argument is evaluated, and its **value is copied** to the corresponding formal parameter for the function.
 - Because the called function works with data copied from the calling function, the original data in the calling function is protected from whatever manipulations the called function applies to the copies.

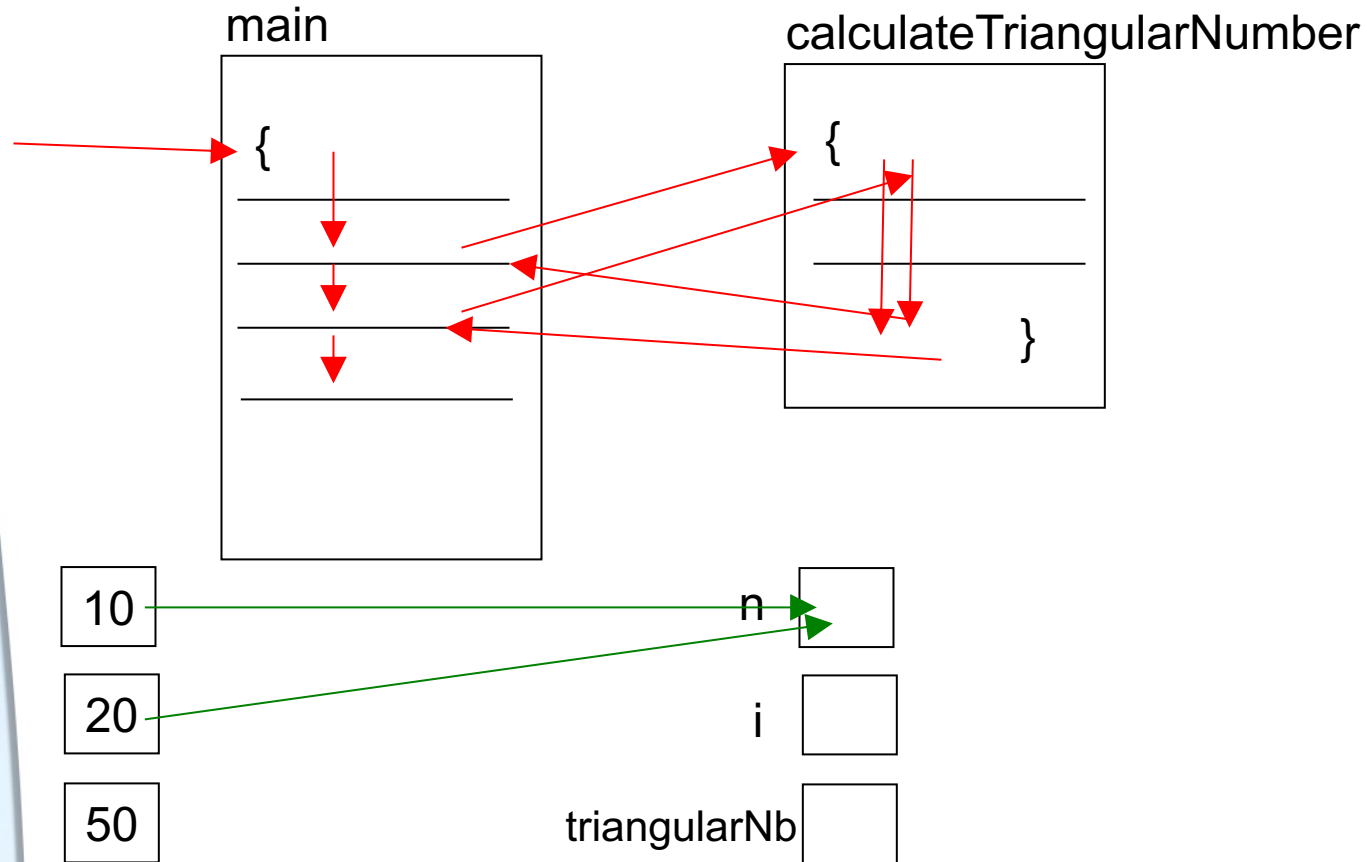
Example: arguments

```
// Function to calculate the nth triangular number
#include <stdio.h>
void calculateTriangularNumber ( int n ) formal argument
{
    int i, triangularNumber = 0; local variables
    for ( i = 1; i <= n; ++i )
        triangularNumber += i;
    printf("Triangular number %i is %i\n",n,triangularNumber);
}
int main (void)
{
    calculateTriangularNumber (10) ; actual argument
    calculateTriangularNumber (20);
    calculateTriangularNumber (50);
    return 0;
}
```

Arguments and local variables

- Variables defined inside a function: *automatic local* variables
 - they are automatically “created” each time the function is called
 - their values are local to the function:
 - The value of a local variable can only be accessed by the function in which the variable is defined
 - Its value cannot be accessed by any other function.
 - If an initial value is given to a variable inside a function, that initial value is assigned to the variable *each* time the function is called.
- Formal parameters: behave like local variables, private to the function.
- **Lifetime**: Period of time when memory location is allocated
- **Scope**: Region of program text where declaration is visible
- Scope: local variables and formal parameters => only in the body of the function
 - Local variable *i* in function `calculateTriangularNumber` is different from a variable *i* defined in another function (including `main`)
 - Formal parameter *n* in function `calculateTriangularNumber` is different from a variable *n* defined in another function

Automatic local variables



Example: scope of local variables

```
#include <stdio.h>
void f1 (float x) {
    int n=6;
    printf("%f \n", x+n);
}
int f2(void) {
    float n=10;
    printf("%f \n",n);
}
int main (void)
{
    int n=5;
    f1(3);
    f2();
    return 0;
}
```

Arguments are passed by copying values !

- In a function call, the actual argument is evaluated, and its **value is copied** to the corresponding formal parameter for the function.
 - Because the called function works with data copied from the calling function, the original data in the calling function is protected from whatever manipulations the called function applies to the copies

Example: arguments

```
#include <stdio.h>
void gcd (int u, int v)
{
    int temp;
    printf ("The gcd of %i and %i is ", u, v);
    while ( v != 0 ) {
        temp = u % v;
        u = v;
        v = temp;
    }
    printf ("%i\n", u);
}
int main (void)
{
    gcd (150, 35);
    gcd (1026, 405);
    gcd (83, 240);
    return 0;
}
```

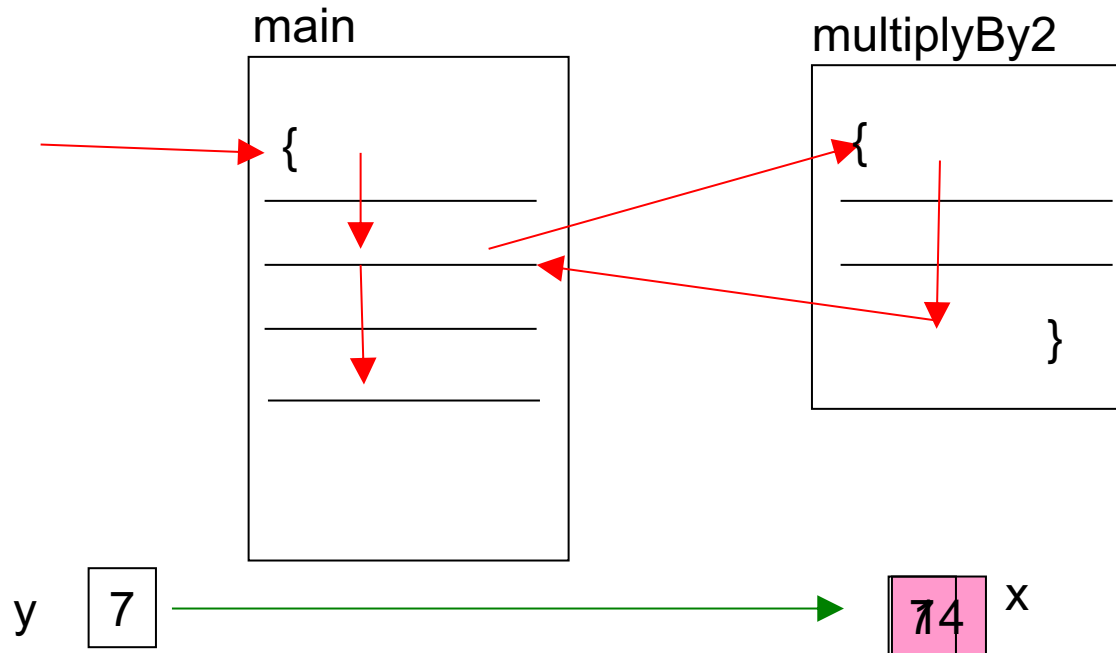
Example: arguments are passed by copying values !

```
#include <stdio.h>
void gcd (int u, int v)
{
    int temp;
    printf ("The gcd of %i and %i is ", u, v);
    while ( v != 0 ) {
        temp = u % v;
        u = v;
        v = temp;
    }
    printf ("%i\n", u);
}
int main (void)
{
    int x=10,y=15;
    gcd (x, y);
    printf("x=%i y=%i \n",x,y);
    return 0;
}
```

The formal parameters u and v are assigned new values in the function

The actual parameters x and y are not changed !

Arguments by copying



Returning function results

- A function in C can optionally return a single value
- `return expression;`
- The value of *expression* is returned to the calling function. If the type of *expression* does not agree with the return type declared in the function declaration, its value is automatically converted to the declared type before it is returned.
- A simpler format for declaring the return statement is as follows:
- `return;`
- Execution of the simple return statement causes program execution to be immediately returned to the calling function. This format can only be used to return from a function that does not return a value.
- If execution proceeds to the end of a function and a return statement is not encountered, it returns as if a return statement of this form had been executed. Therefore, in such a case, no value is returned.
- If the declaration of the type returned by a function is omitted, the C compiler assumes that the function returns an `int` !

Return example

```
void printMessage (void)
{
    printf ("Programming is fun.\n");
    return;
}
```

Example: function result

```
/* Function to find the greatest common divisor of two
nonnegative integer values and to return the result */
#include <stdio.h>
int gcd (int u, int v)
{
    int temp;
    while ( v != 0 ) {
        temp = u % v;
        u = v;
        v = temp;
    }
    return u;
}

int main (void)
{
    int result;
    result = gcd (150, 35);
    printf ("The gcd of 150 and 35 is %i\n", result);
    result = gcd (1026, 405);
    printf ("The gcd of 1026 and 405 is %i\n", result);
    printf ("The gcd of 83 and 240 is %i\n", gcd (83, 240));
    return 0;
}
```

Function declaration

- a function prototype—a declaration that states the return type, the number of arguments, and the types of those arguments.
- Useful mechanism when the called function is defined after the calling function
- The ***prototype*** of the called function is everything the compiler needs in order to be able to ***compile*** the calling function
- In order to produce the ***executable program***, of course that also the whole ***definition*** of the function body is needed, but this occurs later, in the process of ***linking***

Example: function declaration

```
#include <stdio.h>
```

```
void printMessage (void) ;
```

*Function declaration (prototype)
(has to be before the calling function)*

```
int main (void)
```

```
{
```

```
    printMessage ();
```

```
    printMessage ();
```

```
    return 0;
```

```
}
```

Function calls

```
void printMessage (void)
```

```
{
```

```
    printf ("Programming is fun.\n");
```

```
}
```

*Function definition
(can be after the
calling function)*

Global variables

- A **global variable** declaration is made *outside* of any function.
- It does not belong to any particular function. *Any* function in the program can then access the value of that variable and can change its value.
- The primary use of global variables is in programs in which many functions must access the value of the same variable. Rather than having to pass the value of the variable to each individual function as an argument, the function can explicitly reference the variable instead.
- There is a **drawback** with this approach: Because the function explicitly references a particular global variable, the generality of the function is somewhat reduced !
- Global variables do have **default initial values: zero**

Example: global variables

```
#include <stdio.h>

int x;

void f1 (void) {
    x++;
}

void f2 (void) {
    x++;
}

int main(void) {
    x=7;
    f1();
    f2();
    printf("x=%i \n", x);
}
```